

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность ПО-13

Выполнил:

А. В. Шевчук

студент группы ПО-13

Проверил:

А. Д. Кулик

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API.

Общее задание: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

7. Автоматический мониторинг популярных репозиторий

Условие:

Напишите Python-скрипт, который:

1. Запрашивает у пользователя ключевое слово (например, "machine learning", "flask", "fastapi").
2. Использует GitHub API для поиска топ-10 самых популярных репозиторий по этому ключевому слову (сортировка по звездам).
3. Для каждого найденного репозитория получает:
 - ☐ Название
 - ☐ Описание
 - ☐ Количество звезд
 - ☐ Количество форков
 - ☐ Дату последнего коммита
4. Выводит список найденных репозиторий в консоль.
5. Сохраняет результаты в JSON-файл github_top_repos.json.

```
Введите ключевое слово для поиска репозиторий: fastapi
Топ-10 репозиторий по запросу "fastapi":
1. tiangolo/fastapi - ★65,000,    5,400 (Last commit: 2024-03-10)
2. encode/starlette - ★9,800,    1,200 (Last commit: 2024-02-28)
3. ...
Результаты сохранены в github_top_repos.json
```

```
"""Lab 4, task 7.
```

```
Автоматический мониторинг популярных репозиторий GitHub.
"""
```

```
from __future__ import annotations

import json
import os
import sys
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any
from urllib import error, parse, request

API_BASE_URL = "https://api.github.com"
OUTPUT_FILE = Path("github_top_repos.json")

# pylint: disable=too-few-public-methods
@dataclass
class RepositoryInfo:
    """Информация о репозитории GitHub."""

    full_name: str
```

```

description: str
stars: int
forks: int
last_commit_date: str
html_url: str

def build_headers(token: str | None) -> dict[str, str]:
    """Сформировать заголовки для GitHub API."""
    headers = {
        "Accept": "application/vnd.github+json",
        "User-Agent": "spp-p013-lab4-script",
        "X-GitHub-API-Version": "2022-11-28",
    }

    if token:
        headers["Authorization"] = f"Bearer {token}"

    return headers

def github_get_json(url: str, token: str | None) -> Any:
    """Выполнить GET-запрос к GitHub API и вернуть JSON."""
    api_request = request.Request(url, headers=build_headers(token), method="GET")

    try:
        with request.urlopen(api_request, timeout=30) as response:
            raw_data = response.read().decode("utf-8")
            return json.loads(raw_data)
    except error.HTTPError as exc:
        message = exc.read().decode("utf-8", errors="replace")
        raise RuntimeError(f"Ошибка GitHub API: HTTP {exc.code}. Ответ сервера: {message}") from exc
    except error.URLError as exc:
        raise RuntimeError(f"Ошибка сети при обращении к GitHub API: {exc}") from exc

def read_keyword() -> str:
    """Запросить у пользователя ключевое слово."""
    keyword = input("Введите ключевое слово для поиска репозитория: ").strip()

    if not keyword:
        raise ValueError("Ключевое слово не может быть пустым.")

    return keyword

def build_search_url(keyword: str) -> str:
    """Построить URL для поиска топ-10 репозитория по звёздам."""
    query_params = parse.urlencode(
        {
            "q": keyword,
            "sort": "stars",
            "order": "desc",
            "per_page": 10,
            "page": 1,
        }
    )
    return f"{API_BASE_URL}/search/repositories?{query_params}"

```

```

def fetch_top_repositories(keyword: str, token: str | None) -> list[dict[str, Any]]:
    """Получить топ-10 репозиториях по ключевому слову."""
    payload = github_get_json(build_search_url(keyword), token)
    items = payload.get("items", [])

    if not isinstance(items, list):
        raise RuntimeError("GitHub API вернул неожиданный формат данных.")

    return items

def extract_owner_and_repo(full_name: str) -> tuple[str, str]:
    """Разделить полное имя репозитория на owner и repo."""
    parts = full_name.split("/", maxsplit=1)

    if len(parts) != 2:
        raise ValueError(f"Некорректное имя репозитория: {full_name}")

    return parts[0], parts[1]

def fetch_last_commit_date(full_name: str, token: str | None) -> str:
    """Получить дату последнего коммита репозитория."""
    owner, repo_name = extract_owner_and_repo(full_name)
    query = parse.urlencode({"per_page": 1, "page": 1})
    commits_url = f"{API_BASE_URL}/repos/{owner}/{repo_name}/commits?{query}"
    commits = github_get_json(commits_url, token)

    if not isinstance(commits, list) or not commits:
        return "Нет данных"

    commit_data = commits[0].get("commit", {})
    author_data = commit_data.get("author", {})
    return str(author_data.get("date", "Нет данных"))

def normalize_description(description: Any) -> str:
    """Нормализовать описание репозитория."""
    if description is None:
        return "Описание отсутствует"

    text = str(description).strip()
    if not text:
        return "Описание отсутствует"

    return text

def build_repository_info(
    repository_data: dict[str, Any],
    token: str | None,
) -> RepositoryInfo:
    """Построить объект RepositoryInfo из ответа GitHub API."""
    full_name = str(repository_data.get("full_name", "unknown/unknown"))
    description = normalize_description(repository_data.get("description"))
    stars = int(repository_data.get("stargazers_count", 0))
    forks = int(repository_data.get("forks_count", 0))
    html_url = str(repository_data.get("html_url", ""))
    last_commit_date = fetch_last_commit_date(full_name, token)

```

```

return RepositoryInfo(
    full_name=full_name,
    description=description,
    stars=stars,
    forks=forks,
    last_commit_date=last_commit_date,
    html_url=html_url,
)

def collect_repositories_info(
    repositories: list[dict[str, Any]],
    token: str | None,
) -> list[RepositoryInfo]:
    """Собрать итоговую информацию по всем найденным репозиториям."""
    result: list[RepositoryInfo] = []

    for repository_data in repositories:
        repo_info = build_repository_info(repository_data, token)
        result.append(repo_info)

    return result

def print_repositories_report(
    keyword: str,
    repositories: list[RepositoryInfo],
) -> None:
    """Вывести результат в консоль."""
    print(f"\nТоп-10 репозиторияев по запросу \"{keyword}\":")

    for index, repo_info in enumerate(repositories, start=1):
        print(
            f"{index}. {repo_info.full_name} - 🌟 {repo_info.stars}, "
            f"forks: {repo_info.forks} "
            f"(Последний коммит: {repo_info.last_commit_date})"
        )
        print(f"  Описание: {repo_info.description}")
        print(f"  Ссылка: {repo_info.html_url}")

def save_to_json(
    repositories: list[RepositoryInfo],
    output_file: Path = OUTPUT_FILE,
) -> None:
    """Сохранить список репозиторияев в JSON-файл."""
    data = [asdict(repository) for repository in repositories]
    output_file.write_text(
        json.dumps(data, ensure_ascii=False, indent=4),
        encoding="utf-8",
    )

def main() -> None:
    """Точка входа."""
    token = os.getenv("GITHUB_TOKEN")

    try:
        keyword = read_keyword()
        print(f"\nИщем топ-10 репозиторияев по запросу \"{keyword}\"...")

```

```

repositories = fetch_top_repositories(keyword, token)
repositories_info = collect_repositories_info(repositories, token)
print_repositories_report(keyword, repositories_info)
save_to_json(repositories_info)
print(f'\nРезультаты сохранены в "{OUTPUT_FILE}"')
except (RuntimeError, ValueError) as exc:
    print(f'Ошибка: {exc}')
    sys.exit(1)

```

```

if __name__ == "__main__":
    main()

```

```

Введите ключевое слово для поиска репозитиев: react

Ищем топ-10 репозитиев по запросу "react"...

Топ-10 репозитиев по запросу "react":
1. freeCodeCamp/freeCodeCamp - ★444319, forks: 44494 (Последний коммит: 2026-05-07T06:12:12Z)
   Описание: freeCodeCamp.org's open-source codebase and curriculum. Learn math, programming, and computer science for free.
   Ссылка: https://github.com/freeCodeCamp/freeCodeCamp
2. facebook/react - ★244874, forks: 51007 (Последний коммит: 2026-05-07T17:48:57Z)
   Описание: The library for web and native user interfaces.
   Ссылка: https://github.com/facebook/react

```

rep

src

github_top_repos.json

```

1  [
2  {
3      "full_name": "freeCodeCamp/freeCodeCamp",
4      "description": "freeCodeCamp.org's open-source codebase and curriculum. Learn math, programming, and computer science for free.",
5      "stars": 444319,
6      "forks": 44494,
7      "last_commit_date": "2026-05-07T06:12:12Z",
8      "html_url": "https://github.com/freeCodeCamp/freeCodeCamp"
9  },
10 {
11     "full_name": "facebook/react",
12     "description": "The library for web and native user interfaces.",
13     "stars": 244874,
14     "forks": 51007,
15     "last_commit_date": "2026-05-07T17:48:57Z",
16     "html_url": "https://github.com/facebook/react"
17 }

```

Вывод: В ходе работы был изучен API GitHub и реализован Python-скрипт для анализа активности разработчиков. Получены навыки работы с REST API, обработки данных и их визуализации с помощью Matplotlib. Поставленная цель лабораторной работы достигнута.